

Gestión de Información en Aplicaciones Web mediante Objetos del Ambiente y Acceso a Datos: Caso de Estudio Sistema de Gestión de Refuerzos Académicos (SGRA)

Adrián Guizado¹, César Macías¹, Johan Carvajal¹, Melanie Muñoz¹, Mario Zambrano¹

¹ Software - Quinto Semestre Paralelo B, Facultad de Ciencias de la Computación y Diseño Digital,
Universidad Técnica Estatal de Quevedo. Ecuador
{aguizadov, cmaciasv4, jcarvajall, mmunozv6, lzanbranop6}@uteq.edu.ec

Resumen: El presente informe técnico detalla el diseño e implementación del Sistema de Gestión de Refuerzo Académico (SGRA), una plataforma web orientada a optimizar el acompañamiento educativo en la Universidad Técnica Estatal de Quevedo (UTEQ). El proyecto aborda la digitalización de procesos de tutoría mediante una arquitectura API REST desarrollada con Spring Boot, integrando una capa de datos robusta en PostgreSQL. La metodología se centra en la separación de responsabilidades a través de controladores para la gestión de peticiones y respuestas, el uso de objetos de sesión para la seguridad de roles, y la implementación de Spring Data JPA junto con procedimientos almacenados para garantizar la eficiencia en el acceso a la información. Los resultados demuestran un sistema escalable capaz de gestionar flujos de trabajo de estudiantes, docentes y coordinadores, mejorando la trazabilidad del desempeño académico y reduciendo la deserción estudiantil.

Palabras clave: Aplicaciones Web, Spring Boot, PostgreSQL, Gestión de Sesiones, Persistencia de Datos, API REST.

Tabla de contenido

1	Introducción	3
2	Revisión Del Estado Del Arte	3
2.1	Tecnologías de Desarrollo Web y Arquitectura Cliente-Servidor	3
2.2	Gestión del Ciclo de Vida de la Información en Entornos Web	4
2.3	Estrategias de Persistencia y Acceso a Datos	6
3	Sistema Propuesto (Caso De Estudio Sistema De Gestión De Refuerzo Académico)	8
3.1	Descripción General del Sistema de Gestión de Refuerzo Académico	8
3.2	Modelo de Base de Datos (PostgreSQL)	8
3.3	Actores y Roles del Sistema (Estudiantes, Docentes, Administradores)	8
4	Metodología De Desarrollo e Implementación	9
4.1	Configuración del Entorno (Spring Boot + Conexión JDBC/JPA)	9
4.2	Definición de Entidades y Repositorios (Tablas SGRA)	10
4.3	Implementación de la Lógica de Negocio y Objetos del Ambiente.....	11
4.4	Implementación de la Capa de Datos	20
4.4.1	Operaciones CRUD con Spring Data JPA	20
4.4.2	Ejecución de Procedimientos Almacenados (@Procedure)	21
5	Resultados y Discusión	23
5.1	Escenarios de Prueba y Validación por Roles	23
5.2	Análisis de la Gestión de Información en Entornos Multiusuario	26
5.3	Evidencia de la Persistencia de Datos.....	28
6	Conclusiones y Recomendaciones	30
7	Referencias	30
8	Anexos.....	33
8.1	Script SQL (Tablas y Store Procedures).....	33
8.2	Diagrama Relacional	34
8.3	Enlace al Repositorio (GitHub)	34

1 Introducción

El refuerzo académico en las instituciones de educación superior ha pasado de ser un requisito administrativo para convertirse en una herramienta estratégica para cerrar brechas de aprendizaje y asegurar el éxito estudiantil. En la Universidad Técnica Estatal de Quevedo, la gestión tradicional de estas tutorías ha enfrentado limitaciones debido a procesos manuales que dificultan el seguimiento y la generación de indicadores de desempeño en tiempo real. Ante este escenario, surge el Sistema de Gestión de Refuerzo Académico (SGRA) como una solución tecnológica integral que centraliza y automatiza la interacción entre los actores universitarios.

El sistema se fundamenta en estándares de ingeniería de software modernos, adoptando una arquitectura cliente-servidor que garantiza la escalabilidad del servicio y una experiencia de usuario intuitiva. La propuesta técnica se divide en módulos críticos que abarcan desde la configuración de disponibilidad docente hasta el monitoreo gerencial por parte de la coordinación de carrera. Para lograrlo, el desarrollo backend utiliza el framework Spring Boot, permitiendo la creación de endpoints funcionales que gestionan el ciclo de vida de los datos mediante el intercambio de objetos Request y Response.

Finalmente, el proyecto pone especial énfasis en la persistencia y seguridad de la información. Mediante la implementación de Spring Data JPA, el sistema reduce la complejidad del acceso a la base de datos, mientras que el uso de procedimientos almacenados en PostgreSQL optimiza las consultas de alta demanda. Este enfoque técnico no solo asegura la integridad de los datos académicos, sino que posiciona al SGRA como una plataforma robusta para la toma de decisiones basada en datos reales, impactando positivamente en la calidad educativa institucional.

2 Revisión Del Estado Del Arte

2.1 Tecnologías de Desarrollo Web y Arquitectura Cliente-Servidor

La arquitectura cliente-servidor representa el pilar fundamental de las aplicaciones distribuidas modernas, donde la carga de trabajo se reparte entre los proveedores de un recurso o servicio [1], [2], denominados servidores, y los demandantes del servicio, llamados clientes. En el contexto del sistema SGRA, esta arquitectura permite una interacción fluida a través del protocolo HTTP [3], facilitando que el navegador o cliente consuma recursos de una API centralizada sin necesidad de conocer la implementación interna de los datos.

Fundamentos de la Arquitectura Cliente-Servidor (2025): En la actualidad, este modelo ha evolucionado hacia arquitecturas de N-capas [4] que optimizan la entrega de servicios web y móviles mediante una gestión centralizada de recursos. El servidor no solo procesa peticiones, sino que gestiona de manera inteligente el almacenamiento, la seguridad y la concurrencia, permitiendo que múltiples clientes accedan de forma simultánea a la lógica de negocio sin comprometer la integridad de la información [5].

Separación de Incumbencias (Separation of Concerns - SoC): Este principio de ingeniería de software dicta que un sistema debe dividirse en secciones distintivas, donde cada una aborda una responsabilidad específica de la funcionalidad total. En el desarrollo de SGRA, esto se manifiesta en la división clara entre el frontend [6] (interfaz de usuario) y el backend [7] (servidor de aplicaciones).

Tabla 1: Arquitectura Cliente-Servidor

Modularidad y Mantenibilidad	Al aislar la lógica de presentación de la lógica de negocio, se facilita la actualización independiente de componentes y se reduce el riesgo de efectos secundarios no deseados durante el mantenimiento.
Desarrollo en Paralelo	Equipos especializados pueden trabajar de forma concurrente en diferentes módulos (por ejemplo, el desarrollo de controladores REST y el diseño de la vista) sin interferir en el código del otro.
Reusabilidad de Código	Los componentes bien definidos pueden ser reutilizados en distintas partes del sistema o incluso en proyectos externos, lo que acelera el ciclo de vida del desarrollo y reduce costos operativos.

2.2 Gestión del Ciclo de Vida de la Información en Entornos Web

La gestión del ciclo de vida de la información en aplicaciones web modernas [8], [9] determina cómo los datos nacen, se transforman, se almacenan temporalmente y se destruyen durante la interacción del usuario con el sistema. En entornos distribuidos y multiusuario como el SGRA, es imperativo gestionar los ámbitos de manera eficiente para garantizar la integridad de los datos académicos y una experiencia de usuario fluida.

1. Alcance de Petición (Request) y Respuesta (Response)

El ciclo más básico y fundamental es el intercambio de mensajes entre el cliente y el servidor bajo el protocolo HTTP. Este proceso es intrínsecamente sin estado, lo que significa que cada petición es independiente de la anterior.

Protocolo HTTP y Objetos de Transferencia: En el backend [7] del sistema SGRA, se utilizan objetos como `HttpServletRequest` [10] y `HttpServletResponse` para capturar metadatos del cliente (direcciones IP, cabeceras de autenticación) y enviar los resultados del procesamiento.

Gestión de Parámetros (@RequestParam): La información viaja desde la vista al controlador mediante parámetros de consulta [11]. Por ejemplo, al buscar un estudiante por su `idUsuario`, el sistema crea un alcance de petición que vive únicamente durante el tiempo que tarda el servidor en ejecutar la consulta y devolver el ID correspondiente.

Inmutabilidad y Eficiencia [12] y permitiendo que el sistema soporte miles de solicitudes concurrentes de estudiantes y docentes sin saturar la memoria.

2. Persistencia Temporal: Sesiones de Usuario (Session)

Dado que HTTP no recuerda quién es el usuario entre una página y otra, surge la necesidad de la Persistencia Temporal. La sesión es un mecanismo que permite al servidor identificar a un usuario específico a lo largo de múltiples peticiones.

HttpSession y @SessionAttributes: En el proyecto SGRA, las sesiones son críticas para mantener el estado de autenticación [13]. Cuando un Coordinador inicia sesión, sus datos se almacenan en el `HttpSession`, permitiendo que el sistema "recuerde" su rol y permisos mientras navega por los distintos módulos [14] de gestión de tutorías.

Ciclo de Vida de la Sesión: La información en este ámbito persiste desde el momento del login hasta que el usuario cierra explícitamente la sesión o se alcanza un tiempo de inactividad predefinido (timeout). Esto evita que el usuario deba reautenticarse en cada clic, mejorando la usabilidad académica.

Aislamiento de Datos: Cada sesión es privada, un estudiante no puede acceder a los atributos de sesión de otro, lo que garantiza una capa de seguridad lógica esencial para la protección de registros de calificaciones y asistencias.

3. Persistencia Global y Contexto de Página (Application & Page Objects)

Existen datos que trascienden al usuario individual o que deben ser optimizados para la visualización en pantalla.

Ámbito de Aplicación (Application Scope): Se refiere a la información compartida por todos los usuarios del sistema [15]. En SGRA, esto puede incluir configuraciones globales [16], contadores de sistema o cachés de datos estáticos (como catálogos de carreras o modalidades) que se cargan al iniciar el servidor y permanecen disponibles hasta que este se apaga.

Manejo de Objetos de Página (Page Objects): Para evitar el envío masivo de miles de registros de estudiantes hacia el navegador (lo que causaría lentitud), se utiliza la interfaz Page de Spring Data [17].

Paginación Dinámica: Este objeto permite dividir los resultados en bloques manejables (ej. 10 registros por página). El servidor solo procesa y envía el fragmento solicitado, optimizando el ancho de banda y el rendimiento de la base de datos PostgreSQL [18].

Tabla 2: Ámbitos y Ciclo de Vida de los Datos en SGRA

Ámbito (Scope)	Duración / Ciclo de Vida	Propósito Principal en SGRA	Objeto / Anotación Utilizada
Petición (Request)	Instantáneo: Desde que el cliente solicita hasta que recibe la respuesta.	Transferencia de datos simples y ejecución de comandos específicos.	HttpServletRequest, @RequestParam
Respuesta (Response)	Finaliza al entregar el contenido al cliente.	Retornar códigos de estado (200, 201) y datos procesados (JSON)	HttpServletResponse, @ResponseBody
Sesión (Session)	Persistente: Desde el login hasta el logout o inactividad.	Mantener la identidad y el rol del usuario (Estudiante, Docente, Coordinador).	HttpSession, @SessionAttributes
Página (Page)	Específico a la visualización de una lista de datos.	Optimizar el rendimiento al dividir grandes listados de estudiantes en bloques.	Page<T>, Pageable
Aplicación (Application)	Global: Desde que arranca el servidor hasta que se apaga	Almacenar configuraciones globales y constantes compartidas por todo el sistema.	ServletContext, Contexto de Spring

4. Diferencia entre @RequestParam y @ResponseBody

En el desarrollo de aplicaciones web con Spring MVC / Spring Boot [19], el intercambio de información entre el cliente (navegador, Postman, frontend) y el servidor se realiza mediante el

protocolo HTTP [20], el cual se compone principalmente de dos elementos: la petición (request) que envía el cliente y la respuesta (response) que devuelve el servidor [21]. En este contexto, Spring utiliza anotaciones para indicar cómo se deben recibir los datos y cómo se deben entregar las respuestas [22]. Dos anotaciones frecuentes son `@RequestParam` y `@ResponseBody`, las cuales cumplen funciones distintas y se aplican en momentos diferentes del flujo de una solicitud.

La anotación `@RequestParam` se emplea para capturar valores que llegan dentro de la petición, específicamente como parámetros en la URL (query parameters) o datos enviados por formularios [23]. Es decir, permite extraer información que el cliente adjunta a la ruta en formatos como: `?page=0&size=10` o `?nombre=Pedro`. Esta anotación resulta útil cuando se necesitan datos simples (números, textos, banderas booleanas) para controlar búsquedas, filtros, ordenamientos o paginación. Además, `@RequestParam` puede configurarse para que un parámetro sea obligatorio o para asignar un valor por defecto en caso de que el cliente no lo envíe.

Por otro lado, `@ResponseBody` se utiliza para indicar que el valor retornado por un método del controlador debe enviarse directamente como el cuerpo de la respuesta HTTP [24], sin renderizar una vista (como ocurriría en aplicaciones MVC tradicionales con HTML). En términos prácticos, `@ResponseBody` permite que Spring convierta automáticamente los objetos Java retornados (por ejemplo, una entidad `Students`, una lista, o un `Page`) a un formato de intercambio como JSON, utilizando convertidores como Jackson. De esta forma, el cliente recibe una respuesta estructurada y entendible para consumo en APIs REST.

La diferencia esencial es que `@RequestParam` actúa sobre la entrada de datos (obtiene información desde la petición del cliente), mientras que `@ResponseBody` actúa sobre la salida de datos (envía el resultado del servidor al cliente como contenido de la respuesta) [25]. En otras palabras: `@RequestParam` permite leer parámetros enviados por el usuario, y `@ResponseBody` permite devolver datos procesados en forma de JSON/XML sin necesidad de una vista. Cabe destacar que, al trabajar con controladores REST usando la anotación `@RestController`, Spring aplica `@ResponseBody` de manera implícita en todos los métodos, por lo cual muchas veces no es necesario escribirlo explícitamente.

Diferencia principal

Tabla 2. Comparación de `RequestParam`, `ResponseBody`

Aspecto	<code>@RequestParam</code>	<code>@ResponseBody</code>
¿Para qué sirve?	Leer datos del request	Escribir datos en la response
Parte del flujo HTTP	Entrada (Request)	Salida (Response)
¿De dónde obtiene/manda datos?	URL query params / formularios	Cuerpo de respuesta (JSON/XML)
Uso típico	<code>?page=0&size=10</code>	Responder objetos Java como JSON
Se aplica a	Parámetros del método	Al método (o retorno)

2.3 Estrategias de Persistencia y Acceso a Datos

La persistencia de datos representa uno de los desafíos más críticos en el desarrollo de sistemas de información, pues debe garantizar que la información trascienda el ciclo de vida de la aplicación y se almacene de forma segura en un soporte físico. En la arquitectura del sistema SGRA, esta persistencia no se limita al simple almacenamiento, sino que se gestiona a través de

una capa de abstracción basada en Spring Data JPA [26], [27], la cual permite que los desarrolladores interactúen con la base de datos PostgreSQL utilizando objetos de Java en lugar de sentencias SQL directas.

No obstante, para aquellos procesos donde el rendimiento es una prioridad absoluta y se requiere minimizar el tráfico de red, el sistema integra el uso de Procedimientos Almacenados desarrollados en PL/pgSQL. Esta técnica permite delegar lógica de negocio compleja directamente al motor de base de datos, lo que resulta especialmente útil para operaciones de agregación o búsquedas filtradas que involucran múltiples tablas. En el proyecto, funciones como `sp_buscar_estudiante_por_usuario` y `sp_contar_docentes_modalidad` evidencian cómo el procesamiento se realiza de forma local en el servidor de datos, retornando únicamente el valor final necesario al backend de Spring Boot. Esta integración híbrida entre la facilidad de JPA [28] y la potencia de los procedimientos almacenados garantiza que el sistema sea tanto productivo para el desarrollador como eficiente para el usuario final en la Universidad.

1. Mapeo Objeto-Relacional (ORM) y JPA

Fundamentación del Mapeo Objeto-Relacional (ORM): Esta técnica es el puente que permite comunicar el lenguaje de programación Java con el motor de base de datos PostgreSQL [18], traduciendo las clases del sistema hacia tablas relacionales de forma automática. En el proyecto SGRA, esto asegura que los datos de estudiantes y docentes se manejen como objetos dentro del código, manteniendo la integridad de la información académica sin necesidad de escribir consultas manuales complejas para cada acción.

Implementación de JPA y el Patrón Repository: El uso de Java Persistence API (JPA) [29] a través de Spring Data permite que el sistema gestione la persistencia mediante interfaces que heredan de `JpaRepository`. Esta estrategia es fundamental porque proporciona de manera nativa todas las operaciones CRUD [30] (Crear, Leer, Actualizar y Eliminar), lo que reduce el código repetitivo y permite que el equipo se enfoque en la lógica del refuerzo académico en lugar de la infraestructura de datos.

2. Optimización mediante Procedimientos Almacenados

Para procesos que requieren un alto rendimiento [31], el sistema delega lógica directamente al motor PostgreSQL a través de rutinas en PL/pgSQL [32]. Al ejecutar funciones como `sp_buscar_estudiante_por_usuario` y `sp_contar_docentes_modalidad` en el lado del servidor de base de datos, se reduce drásticamente el tráfico de red y se mejora el tiempo de respuesta, ya que solo se envía el resultado final a la aplicación.

Para el encapsulamiento y seguridad del uso de procedimientos almacenados actúa como una capa de protección adicional para la base de datos. Al invocar estas funciones mediante la anotación `@Query` en los repositorios de Spring [33], el sistema garantiza que el acceso a las tablas `TbEstudiantes` y `TbDocentes` esté controlado, minimizando riesgos de seguridad y asegurando que solo se expongan los datos estrictamente necesarios para el rol del usuario.

La combinación de JPA [34] y procedimientos, el sistema SGRA está preparado para manejar un volumen creciente de datos universitarios de forma escalable. Esta arquitectura híbrida permite que las transacciones sean seguras y consistentes, asegurando que el registro de una tutoría o una asistencia no se pierda ni se duplique, incluso en entornos de alta concurrencia de usuarios.

3 Sistema Propuesto (Caso De Estudio Sistema De Gestión De Refuerzo Académico)

3.1 Descripción General del Sistema de Gestión de Refuerzo Académico

El Sistema de Gestión de Refuerzo Académico (SGRA) se presenta como una solución tecnológica integral diseñada para optimizar los procesos de acompañamiento educativo en la Universidad Técnica Estatal de Quevedo (UTEQ). La plataforma aborda el refuerzo académico no solo como un requisito administrativo, sino como una herramienta para cerrar brechas de aprendizaje, mejorar el desempeño en asignaturas críticas y reducir el riesgo de deserción estudiantil.

El sistema digitaliza el flujo de trabajo actual, superando las limitaciones de la gestión manual o informal que históricamente ha dificultado la trazabilidad y la generación de indicadores. A través de una arquitectura web centralizada, el SGRA permite la organización eficiente de solicitudes de tutoría, la gestión de la disponibilidad docente en tiempo real y el seguimiento detallado de las sesiones impartidas (presenciales o virtuales). La propuesta se fundamenta en estándares de ingeniería de software y un diseño centrado en el usuario, garantizando la consistencia de los datos, la escalabilidad del servicio y una experiencia de uso intuitiva para toda la comunidad universitaria.

3.2 Modelo de Base de Datos (PostgreSQL)

Para garantizar la integridad y la persistencia de la información del SGRA, se diseñó un modelo relacional normalizado implementado en el motor de base de datos PostgreSQL. El esquema se centra en la gestión eficiente de los diferentes tipos de usuarios y sus actividades académicas.

El modelo consta de las siguientes tablas principales:

1. **TbEstudiantes:** Almacena la información de los alumnos beneficiarios, vinculando su perfil de usuario (idusuario) con su carrera (idcarrera) y manteniendo su estado de actividad.
2. **TbDocentes:** Registra a los profesores habilitados para impartir tutorías, incluyendo su modalidad de contratación (idmodalidad) y disponibilidad.
3. **TbCoordinaciones:** Gestiona los roles administrativos encargados de supervisar el proceso de refuerzo académico por carrera.

Todas las tablas mantienen una relación directa con una tabla maestra de usuarios a través de llaves foráneas.

Se puede visualizar la representación gráfica de las tablas y sus relaciones en el Diagrama Entidad-Relación adjunto en el Anexo B.

3.3 Actores y Roles del Sistema (Estudiantes, Docentes, Administradores)

Para esta actividad se estará utilizando el módulo de usuarios, este sistema identifica tres actores principales, los cuales se conforman por: Estudiante, Docente y Coordinación

Estudiante: Es el beneficiario principal del sistema. Su interacción se centra en la generación de solicitudes de refuerzo académico seleccionando asignaturas y temas específicos donde requiere apoyo.

Docente: Actúa como el proveedor del servicio académico. Su rol en el sistema inicia con la configuración de su disponibilidad horaria (TbDisponibilidadDocente), definiendo las franjas en las que puede atender estudiantes. Tiene la responsabilidad de gestionar las solicitudes entrantes (aceptar, rechazar o reprogramar), registrar la ejecución de las tutorías en (TbRefuerzoRealizado) y tomar la asistencia de los participantes, alimentando así los indicadores de desempeño académico.

Coordinador de Carrera: Este actor posee una visión gerencial del proceso. Encargado de supervisar la correcta ejecución de las tutorías dentro de su carrera (TbCoordinaciones), el coordinador tiene acceso a módulos de reporte que le permiten monitorear estadísticas clave: temas más solicitados, docentes con mayor carga de tutorías y estudiantes en riesgo académico. Su función es vital para la toma de decisiones basada en los datos generados por el SGRA.

4 Metodología De Desarrollo e Implementación

4.1 Configuración del Entorno (Spring Boot + Conexión JDBC/JPA)

Para el desarrollo del backend del sistema SGRA, se configuró un proyecto basado en el framework Spring Boot, gestionando las dependencias a través de Maven. Esta elección garantiza una arquitectura robusta y facilita la inyección de dependencias necesaria para la escalabilidad del software.

La conexión con la base de datos relacional se estableció utilizando el driver de PostgreSQL (org.postgresql.Driver). La configuración de las propiedades de conexión se definió en el archivo application.properties, donde se especificaron las credenciales, la cadena de conexión JDBC y el dialecto de Hibernate para asegurar la compatibilidad con las funciones específicas de la base de datos.

Dependencias clave utilizadas (pom.xml):

- **spring-boot-starter-web:** Para el despliegue del contenedor Servlet y la creación de la API REST.
- **spring-boot-starter-data-jpa:** Para la implementación de la capa de persistencia y el manejo de repositorios.
- **postgresql:** Driver JDBC para la comunicación con el motor de base de datos.

application.properties:

```
spring.application.name=SGRA
spring.datasource.url=jdbc:postgresql://localhost:5432/sgraweb
spring.datasource.username= postgres
spring.datasource.password=P123
spring.datasource.driver-class-name= org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql = true
spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.PostgreSQLDialect
server.port=8080
```

4.2 Definición de Entidades y Repositorios (Tablas SGRA)

Siguiendo el patrón de Mapeo Objeto-Relacional (ORM), se transformó el modelo lógico de la base de datos (tablas) en un modelo orientado a objetos (clases). Se utilizaron las anotaciones estándar de JPA (Jakarta Persistence API) para vincular las entidades con el esquema relacional definido en PostgreSQL.

Mapeo de Entidades (@Entity)

Cada tabla del sistema SGRA se representó mediante una clase, utilizando anotaciones para definir la estructura de datos:

- **@Entity y @Table:** Indican que la clase representa una tabla específica (ej. Students → tbestudiantes).
- **@Id y @GeneratedValue:** Definen la clave primaria y su estrategia de autoincremento (IDENTITY), alineándose con las columnas generated by default as identity definidas en el script SQL.
- **@Column:** Mapea los atributos de la clase con las columnas de la base de datos.

Las entidades principales implementadas fueron:

1. **Students:** Mapea la tabla tbestudiantes, gestionando la relación entre el usuario, su carrera y su estado académico.
2. **Teaching:** Representa la tabla tbdocentes, incluyendo la modalidad de contratación.
3. **Coordination:** Vincula a los usuarios administrativos con sus respectivas carreras (tbcoordinaciones).
4. **Users y Rol:** Gestionan la seguridad y autenticación del sistema.

Implementación de Repositorios (Repository)

Para la manipulación de estas entidades, se crearon interfaces que extienden de JpaRepository<T, ID>. Esto proporciona automáticamente los métodos estándar para operaciones CRUD (save, findAll, findById, delete) sin necesidad de escribir sentencias SQL manuales.

Adicionalmente, se definieron consultas personalizadas utilizando la anotación @Query para invocar los procedimientos almacenados, como sp_buscar_estudiante_por_usuario y sp_contar_docentes_modalidad, optimizando así el rendimiento en consultas complejas.

Clase students

```
package com.webSGRA.SGRA.Entities;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "tbstudents")
@AllArgsConstructor
@NoArgsConstructor
@Data
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
@NoArgsConstructor
```

```

@Table(name = "tbestudiantes")
public class Students{
    @Id
    @Column(name = "idestudiante")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @EqualsAndHashCode.Include
    private Integer studentId;

    @Column(name = "idusuario", nullable = false)
    private Integer userId;

    @Column(name = "idcarrera", nullable = false)
    private Integer careerId;

    @Column(name = "estado", nullable = false, columnDefinition = "boolean default
true")
    private Boolean state = true;
}

```

Clase studentsRepository

```

package com.webSGRA.SGRA.Repositories;

import com.webSGRA.SGRA.Entities.Students;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentsRepository extends JpaRepository<Students, Integer> {

    Students findByUserId(Integer userId);

    @Query(value = "SELECT sp buscar estudiante por usuario(?1)", nativeQuery = true)
    Integer obtenerIdEstudiantePorUsuario(Integer idUsuario);
}

```

4.3 Implementación de la Lógica de Negocio y Objetos del Ambiente

Controlador de pruebas: TestController

Se creó un controlador de propósito demostrativo para validar endpoints simples y evidenciar el uso de Request/Response.

Ruta base del controlador: /api/test

Endpoints implementados:

1. **Consultar ID de estudiante por usuario**
GET /api/test/estudiante?idUsuario=1
Descripción: Obtiene el ID del estudiante asociado al usuario enviado por parámetro.
Entrada: idUsuario (RequestParam)
Salida: Mensaje de texto con el ID encontrado.
2. **Contar docentes por modalidad**
GET /api/test/docentes/conteo?modalidad=1
Descripción: Retorna el total de docentes según la modalidad.
Entrada: modalidad (RequestParam)
Salida: Mensaje con el número total.

Código de TestController

```
package com.webSGRA.SGRA.Controllers;

import com.webSGRA.SGRA.Service.SgraService;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/test")
@RequiredArgsConstructor
public class TestController {

    private final SgraService sgraService;

    // URL: http://localhost:8080/api/test/estudiante?idUsuario=1
    @GetMapping("/estudiante")
    public String buscarEstudiante(
        @RequestParam Integer idUsuario,
        HttpServletRequest request,
        HttpServletResponse response
    ) {
        String ip = request.getRemoteAddr();
        String userAgent = request.getHeader("User-Agent");

        Integer idEst = sgraService.obtenerIdEstudiante(idUsuario);

        response.setHeader("X-Cliente-IP", ip);
        response.setHeader("X-User-Agent", userAgent);

        return "El ID del Estudiante para el usuario " + idUsuario + " es: " + idEst;
    }

    @GetMapping("/docentes/conteo")
    public String contarDocentes(
        @RequestParam Integer modalidad,
        HttpServletRequest request,
        HttpServletResponse response
    ) {
        response.setHeader("X-Metodo", request.getMethod());

        Integer total = sgraService.contarDocentes(modalidad);
        return "Total de docentes en modalidad " + modalidad + ": " + total;
    }
}
```

Controlador principal: StudentsApiController

Se desarrolló un controlador REST para exponer operaciones sobre la entidad Students mediante endpoints API.

Ruta base del controlador: /api/students

Endpoints implementados:

1. Listar estudiantes con paginación

GET /api/students?page=0&size=10

Descripción: Devuelve un objeto Page<Students> con resultados paginados.

Salida: JSON con lista, total de elementos, total de páginas, página actual, etc.

2. Buscar estudiante por ID

GET /api/students/{id}

Descripción: Retorna un estudiante por su ID.

Resultado esperado:

- 200 OK si existe.
- 404 NOT FOUND si no existe.

3. Crear estudiantes

POST /api/students

Descripción: Registra un nuevo estudiante usando el cuerpo JSON (@RequestBody).

Resultado esperado:

➤ **201 CREATED** con el estudiante creado.

4. Eliminar estudiantes

DELETE /api/students/{id}

Descripción: Elimina el estudiante asociado al ID indicado.

Resultado esperado:

➤ **204 NO CONTENT** si se ejecuta correctamente.

Código de StudentApiController;

```
package com.webSGRA.SGRA.Controllers;

import com.webSGRA.SGRA.Entities.Students;
import com.webSGRA.SGRA.Service.SgraService;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
public class StudentsApiController {

    private final SgraService sgraService;

    // GET /api/students?page=0&size=10
    @GetMapping
    public ResponseEntity<Page<Students>> listar(
        @PageableDefault(size = 10) Pageable pageable,
        HttpServletRequest request,
        HttpServletResponse response
    ) {
        // Usando Request (ejemplo)
        String ip = request.getRemoteAddr();
        String userAgent = request.getHeader("User-Agent");

        Page<Students> page = sgraService.listarEstudiantes(pageable);

        // Usando Response: headers con info de paginación
        response.setHeader("X-Cliente-IP", ip);
        response.setHeader("X-User-Agent", userAgent);
        response.setHeader("X-Total-Items", String.valueOf(page.getTotalElements()));
        response.setHeader("X-Total-Pages", String.valueOf(page.getTotalPages()));
        response.setHeader("X-Page-Number", String.valueOf(page.getNumber()));
        response.setHeader("X-Page-Size", String.valueOf(page.getSize()));

        return ResponseEntity.ok(page);
    }

    // GET /api/students/5
    @GetMapping("/{id}")
    public ResponseEntity<?> buscarPorId(
        @PathVariable Integer id,
        HttpServletResponse response
    ) {
        return sgraService.buscarEstudiantePorId(id)
            .<ResponseEntity<?>>map(s -> {
                response.setHeader("X-Resultado", "ENCONTRADO");
                return ResponseEntity.ok(s);
            })
    }
}
```

```

        .orElseGet(() -> {
            response.setHeader("X-Resultado", "NO_ENCONTRADO");
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body("Student no encontrado con id=" + id);
        });
    }

    // POST /api/students
    @PostMapping
    public ResponseEntity<Students> crear(
        @RequestBody Students body,
        HttpServletRequest request,
        HttpServletResponse response
    ) {
        // Request usado (ejemplo)
        response.setHeader("X-Operacion", "CREAR");
        response.setHeader("X-Cliente-IP", request.getRemoteAddr());

        Students creado = sgraService.guardarEstudiante(body);

        // Response status 201
        response.setStatus(HttpServletResponse.SC_CREATED);
        return ResponseEntity.status(HttpStatus.CREATED).body(creado);
    }

    // DELETE /api/students/5
    @DeleteMapping("/{id}")
    public ResponseEntity<?> eliminar(
        @PathVariable Integer id,
        HttpServletResponse response
    ) {
        sgraService.eliminarEstudiante(id);
        response.setHeader("X-Operacion", "ELIMINAR");
        return ResponseEntity.noContent().build();
    }
}

```

Programar el uso de objetos Request (HttpServletRequest) y Response

Uso de HttpServletRequest

Se utilizó HttpServletRequest para capturar información del cliente que realiza la petición, como:

- **IP del cliente:** request.getRemoteAddr()
- **User-Agent del navegador:** request.getHeader("User-Agent")
- **Método HTTP utilizado:** request.getMethod()

Esto permite tener trazabilidad del consumo de la API y evidencia de que el backend puede leer metadatos de la solicitud.

Uso de HttpServletResponse

Se utilizó HttpServletResponse para enviar información adicional en cabeceras (headers), por ejemplo:

- X-Cliente-IP
- X-User-Agent
- X-Operacion (CREAR / ELIMINAR / ACTUALIZAR)
- X-Total-Items, X-Total-Pages, etc. (para paginación)

De esta forma, además del cuerpo de respuesta (JSON o texto), el servidor envía metadatos útiles para el cliente.

Gestionar el objeto Page

Para la visualización de listados en la interfaz (vista), se implementó la paginación mediante el objeto:

- Page<Students>
- Pageable (con @PageableDefault(size = 10))

Ventajas de usar Page:

- Evita traer todos los registros de una sola vez.
- Mejora rendimiento y tiempo de respuesta.
- Permite navegación por páginas desde el frontend (siguiente/anterior).
- Incluye información adicional: total de registros, total de páginas, número de página, tamaño, etc.

Además, se enviaron valores del objeto Page en headers para facilitar consumo desde el cliente:

- Total de elementos
- Total de páginas
- Número de página actual
- Tamaño de página

Manejo de Sesiones y Atributos (HttpSession, @SessionAttributes)

El manejo de sesiones es un componente fundamental en las aplicaciones web modernas, ya que el protocolo HTTP es inherentemente sin estado. Para solventar esta limitación, frameworks como Spring Boot proporcionan mecanismos que permiten conservar información del usuario entre múltiples solicitudes, facilitando la continuidad de los procesos y mejorando la experiencia de uso del sistema [35], [36].

En el sistema desarrollado, se empleó el objeto HttpSession para almacenar información asociada al usuario durante su interacción con la plataforma, como identificadores y roles, permitiendo que los trámites administrativos se mantengan consistentes a lo largo de la sesión activa. El uso de sesiones reduce la necesidad de reenviar información sensible en cada solicitud HTTP, lo que contribuye a una mayor eficiencia y seguridad del sistema [37].

Adicionalmente, se utilizó la anotación @SessionAttributes en los controladores de Spring MVC para mantener atributos específicos del modelo durante el ciclo de vida de la sesión del usuario. Este enfoque es recomendado cuando se requiere preservar datos relacionados con flujos de trabajo sin recurrir a un manejo manual complejo de la sesión, favoreciendo una separación clara entre la lógica de presentación y la lógica de negocio [35], [36].

Diversos estudios destacan que una gestión adecuada del estado de sesión es crítica para evitar vulnerabilidades comunes en aplicaciones web, como el uso indebido de cookies de sesión o la persistencia innecesaria de información sensible [38], [39]. Por tanto, la implementación correcta del manejo de sesiones refuerza la seguridad, la integridad de los datos y la estabilidad del sistema. Para el manejo de sesiones se usaron las siguientes.

En primer lugar, se creó un directorio donde se guardan los archivos dtos, en el cual se crearon los siguientes componentes de la capa de transferencia de datos.

LoginRequest:

```
package com.webSGRA.SGRA.Dtos;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.apache.commons.lang3.StringUtils;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class LoginRequest {

    private String userName;
    private String password;
}
```

En este componente lo que se quiere lograr es recibir el usuario y la contraseña del usuario que desea iniciar sesión.

El siguiente componente que se creó fue UserSession.

```
package com.webSGRA.SGRA.Dtos;

import lombok.AllArgsConstructor;
import lombok.Getter;

@Getter
@AllArgsConstructor
public class UserSession {
    private Integer idusuario;
    private String userName;
    private String rol;
}
```

Lo que se piensa manejar con este componente es representar los datos de la sesión en este caso datos del usuario autenticado.

```
package com.webSGRA.SGRA.Dtos;

import lombok.Data;

@Data
public class UpdatePasswordRequest {
    private Integer idUsuario;
    private String newPassword;
}
```

En este componente de transferencia de datos va a tener el id del usuario y la nueva contraseña que se desea actualizar.

Después de eso se crearon dos interfaces en el directorio de Repository las cuales son:

RolRepository

```
package com.webSGRA.SGRA.Repositories;

import com.webSGRA.SGRA.Entities.Rol;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface RolRepository extends JpaRepository<Rol, Integer> {
```

```
Optional<Rol> findByNombre(String nombre);
}
```

La interface RolRepository extiende JpaRepository, lo que permite realizar operaciones CRUD sobre la entidad Rol sin necesidad de implementar métodos manualmente. Además, define el método findByNombre, que permite recuperar un rol específico utilizando su nombre, aprovechando las consultas derivadas de Spring Data JPA.

UserRepository

```
package com.webSGRA.SGRA.Repositories;

import com.webSGRA.SGRA.Entities.Users;
import org.apache.catalina.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<Users,Integer> {

    Optional<Users> findByUsername(String username);
}
```

La interface UserRepository extiende JpaRepository, lo que permite realizar operaciones CRUD sobre la entidad Users sin necesidad de implementar métodos manuales. Además, define el método findByUsername, que permite recuperar un usuario específico a partir de su nombre de usuario, utilizando las consultas derivadas que proporciona Spring Data JPA.

Una vez tenemos la comunicación con la Base de datos se creó el componente de Service que nos permitirá que podamos realizar el proceso para la autenticación en el sistema. El componente se lo definió como AuthService.

```
package com.webSGRA.SGRA.Service;

import com.webSGRA.SGRA.Dtos.LoginRequest;
import com.webSGRA.SGRA.Dtos.UserSession;
import com.webSGRA.SGRA.Entities.Users;
import com.webSGRA.SGRA.Repositories.UserRepository;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;
import org.jspecify.annotations.NonNull;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class AuthService {
    private final UserRepository userRepository;

    public UserSession login(@NonNull LoginRequest req) {
        Users u = userRepository.findByUsername(req.getUserName())
            .orElseThrow(() -> new RuntimeException("Usuario no existe"));

        if (Boolean.FALSE.equals(u.getEstado())) {
            throw new RuntimeException("Usuario inactivo");
        }

        if (!u.getPassword().equals(req.getPassword())) {
            throw new RuntimeException("Credenciales inválidas");
        }

        return new UserSession(
            u.getIdusuario(),
            u.getUsername(),
            u.getRol().getNombre()
        );
    }

    public void updatePassword(Integer idusuario, String newPassword) {
        Users user = userRepository.findById(idusuario)

```

```

        .orElseThrow(() -> new RuntimeException("Usuario no encontrado"));

        user.setPassword(newPassword); // DEMO simple

        userRepository.save(user);
    }
}

```

La clase AuthService pertenece a la capa de servicio y se encarga de gestionar la lógica de autenticación del sistema. Implementa procesos como el inicio de sesión y la actualización de contraseñas, validando la existencia, estado y credenciales del usuario, y retornando información controlada mediante DTOs, para garantizar la seguridad y correcta separación de responsabilidades.

El siguiente componente que se creo fue el AuthController el cual va a manejar las peticiones HTTP.

```

package com.webSGRA.SGRA.Controllers;

import com.webSGRA.SGRA.Dtos.LoginRequest;
import com.webSGRA.SGRA.Dtos.UpdatePasswordRequest;
import com.webSGRA.SGRA.Dtos.UserSession;
import com.webSGRA.SGRA.Entities.Users;
import com.webSGRA.SGRA.Repositories.UserRepository;
import com.webSGRA.SGRA.Service.AuthService;
import jakarta.servlet.ServletContext;
import jakarta.servlet.http.HttpSession;
import lombok.RequiredArgsConstructor;
import org.apache.catalina.User;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class AuthControllers {

    private final AuthService authService;
    private final ServletContext servletContext;

    private static final String SESSION_USER_KEY = "USER";
    private final UserRepository userRepository;

    @PostMapping("/login")
    public ResponseEntity<> login(@RequestBody LoginRequest req, HttpSession session) {
        UserSession user = authService.login(req);

        //Guardar usuario en sesión
        session.setAttribute(SESSION_USER_KEY, user);

        //Application scope (global)
        Integer totalLogins = (Integer) servletContext.getAttribute("totalLogins");
        if (totalLogins == null) totalLogins = 0;
        totalLogins++;
        servletContext.setAttribute("totalLogins", totalLogins);
        servletContext.setAttribute("lastLoginUser", user.getUserName());

        return ResponseEntity.ok(Map.of(
            "message", "Login OK",
            "user", user,
            "sessionId", session.getId()
        ));
    }

    @GetMapping("/me")
    public ResponseEntity<> me(HttpSession session) {
        UserSession user = (UserSession) session.getAttribute(SESSION_USER_KEY);
        if (user == null) {
            return ResponseEntity.status(401).body(Map.of("error", "No hay sesión activa"));
        }
    }
}

```

```

    }
    return ResponseEntity.ok(Map.of("user", user));
}

@PostMapping("/logout")
public ResponseEntity<?> logout(HttpSession session) {
    session.invalidate();
    return ResponseEntity.ok(Map.of("message", "Logout OK"));
}

@PutMapping("/updatePassword")
public ResponseEntity<?> updatePassword(@RequestBody UpdatePasswordRequest req,
                                       HttpSession session) {

    UserSession sessionUser = (UserSession) session.getAttribute(SESSION_USER_KEY);
    if (sessionUser == null) {
        return ResponseEntity.status(401).body(Map.of("error", "No hay sesión
activa"));
    }

    if (!"ADMIN".equalsIgnoreCase(sessionUser.getRol())) {
        return ResponseEntity.status(403).body(Map.of("error", "Acceso denegado:
solo administradores"));
    }

    if (req.getIdUsuario() == null || req.getIdUsuario() <= 0) {
        return ResponseEntity.badRequest().body(Map.of("error", "idusuario
inválido"));
    }
    if (req.getNewPassword() == null || req.getNewPassword().trim().length() < 4) {
        return ResponseEntity.badRequest().body(Map.of("error", "La contraseña debe
tener al menos 4 caracteres"));
    }

    authService.updatePassword(req.getIdUsuario(), req.getNewPassword());

    return ResponseEntity.ok(Map.of(
        "message", "Contraseña actualizada",
        "updatedUserId", req.getIdUsuario(),
        "performedBy", sessionUser.getUserName()
    ));
}
}

```

AuthControllers es un controlador REST encargado de gestionar la autenticación del sistema. Proporciona endpoints para login, consulta del usuario autenticado (/me), logout y actualización de contraseña. Utiliza HttpSession para mantener la sesión del usuario y ServletContext para almacenar estadísticas globales de logins. La lógica de negocio se delega al componente AuthService, manteniendo separación de responsabilidades.

Ámbito de Aplicación (Application Scope)

El ámbito de aplicación (Application Scope) permite compartir información global entre todos los usuarios de una aplicación web durante el ciclo de vida del servidor. En aplicaciones Java basadas en la especificación Servlet, este comportamiento se implementa mediante el objeto ServletContext, el cual proporciona un espacio común para almacenar atributos accesibles desde cualquier componente del sistema [35], [37].

En el sistema desarrollado, se utilizó ServletContext para gestionar datos globales como configuraciones generales y parámetros del sistema que deben estar disponibles para todos los usuarios, independientemente de su sesión. Este enfoque evita la duplicación innecesaria de información y garantiza la consistencia de los datos compartidos en toda la aplicación [35], [36].

Spring Boot permite una integración directa con la API de servlets, facilitando el uso de ServletContext dentro de controladores y servicios cuando se requiere un control explícito del

ámbito de aplicación. Este mecanismo resulta especialmente útil para aplicaciones académicas y administrativas donde múltiples usuarios acceden simultáneamente a los mismos recursos globales [35].

El uso adecuado del Application Scope contribuye a mejorar el rendimiento del sistema y optimizar el uso de memoria, ya que los objetos compartidos se inicializan una sola vez y permanecen disponibles durante todo el tiempo de ejecución de la aplicación. Este enfoque es ampliamente recomendado en aplicaciones empresariales desarrolladas con Spring Boot [36].

Ahora con lo que se respecta al uso de Application scope se realizó un AppController

```
package com.webSGRA.SGRA.Controllers;

import jakarta.servlet.ServletContext;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/app")
@RequiredArgsConstructor
public class AppController {
    private final ServletContext servletContext;

    @GetMapping("/stats")
    public ResponseEntity<?> stats() {
        Integer totalLogins = (Integer) servletContext.getAttribute("totalLogins");
        String lastLoginUser = (String) servletContext.getAttribute("lastLoginUser");

        if (totalLogins == null) totalLogins = 0;

        return ResponseEntity.ok(Map.of(
            "totalLogins", totalLogins,
            "lastLoginUser", lastLoginUser
        ));
    }
}
```

La clase AppController es un controlador REST encargado de exponer estadísticas globales del sistema. Utiliza ServletContext para acceder al application scope, permitiendo consultar información compartida por todos los usuarios, como el número total de inicios de sesión y el último usuario autenticado.

4.4 Implementación de la Capa de Datos

4.4.1 Operaciones CRUD con Spring Data JPA

Para la persistencia de datos del SGRA, se implementó el módulo Spring Data JPA, el cual proporciona una capa de abstracción sobre el almacenamiento de datos relacional [40]. Esta tecnología permitió reducir significativamente el código repetitivo (boilerplate) asociado a las operaciones de base de datos [41].

La implementación se basó en el patrón de diseño Repository. Se definieron interfaces Java que extienden de JpaRepository<T, ID>, donde T representa la entidad JPA e ID el tipo de dato de su clave primaria [41]. Al heredar de esta interfaz, el framework genera automáticamente la implementación de las operaciones CRUD (Create, Read, Update, Delete) básicas en tiempo de ejecución [42].

Implementación en el Proyecto:

Para gestionar la información de los actores del sistema, se crearon las siguientes interfaces de repositorio:

- **Repositorio de Estudiantes:** Permite la gestión de la entidad Students (mapeada a la tabla tbestudiantes).

```
package com.webSGRA.SGRA.Repositories;

import com.webSGRA.SGRA.Entities.Students;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentsRepository extends JpaRepository<Students, Integer> {

    Students findByUserId(Integer userId);

    @Query(value = "SELECT public.sp_buscar_estudiante_por_usuario(?1)", nativeQuery = true)
    Integer obtenerIdEstudiantePorUsuario(Integer idUsuario);
}
```

- **Repositorio de Docentes:** Gestiona la entidad Teaching (mapeada a la tabla tbdocentes).

```
package com.webSGRA.SGRA.Repositories;

import com.webSGRA.SGRA.Entities.Teaching;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;

@Repository
public interface TeachingRepository extends JpaRepository<Teaching, Integer> {

    @Query(value = "SELECT sp_contar_docentes_modalidad(?1)", nativeQuery = true)
    Integer contarDocentesPorModalidad(Integer idModalidad);
}
```

- **Repositorio de Coordinador:** Gestiona la entidad Coordination (mapeada a la tabla tbcoordinaciones), facilitando la administración de los responsables por carrera.

```
package com.webSGRA.SGRA.Repositories;

import com.webSGRA.SGRA.Entities.Coordination;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface CoordinationRepository extends JpaRepository<Coordination, Integer>
{
}
```

4.4.2 Ejecución de Procedimientos Almacenados (@Procedure)

En cumplimiento con los requisitos de la práctica experimental para delegar lógica de negocio a la base de datos, se implementaron rutinas en PL/pgSQL (PostgreSQL) que actúan como procedimientos almacenados [43]. Teniendo como objetivo optimizar el rendimiento, reduciendo el tráfico de red entre el servidor de aplicaciones y el servidor de base de datos [44].

Se definieron los siguientes procedimientos almacenados clave:

- **sp_buscar_estudiante_por_usuario(p_id_usuario):** Encapsula la lógica de búsqueda para recuperar el identificador único de un estudiante a partir de sus credenciales de usuario, garantizando que solo se expongan los datos necesarios.

```
create function sp_buscar_estudiante_por_usuario(p_id_usuario integer) returns
integer
language plpgsql
as
$$
DECLARE
    v_id_estudiante INT;
BEGIN
    SELECT IdEstudiante INTO v_id_estudiante
    FROM TbEstudiantes
    WHERE IdUsuario = p_id_usuario;

    RETURN v_id_estudiante;
END;
$$;

alter function sp_buscar_estudiante_por_usuario(integer) owner to postgres;
```

- **sp_contar_docentes_modalidad(p_id_modalidad):** Realiza una operación de agregación (conteo) del personal docente activo filtrado por su modalidad de trabajo, centralizando la lógica estadística en la base de datos.

```
create function sp_contar_docentes_modalidad(p_id_modalidad integer) returns integer
language plpgsql
as
$$
DECLARE
    v_total INT;
BEGIN
    SELECT COUNT(*) INTO v_total
    FROM TbDocentes
    WHERE IdModalidad = p_id_modalidad AND Estado = TRUE;

    RETURN v_total;
END;
$$;

alter function sp_contar_docentes_modalidad(integer) owner to postgres;
```

Implementación en la Capa de Datos: Para la invocación de estos procedimientos desde la aplicación Spring Boot, se configuraron métodos personalizados en las interfaces StudentsRepository y TeachingRepository.

Metodo implementado en StudentsRepository:

```
@Query(value = "SELECT public.sp_buscar_estudiante_por_usuario(?1)", nativeQuery = true)
Integer obtenerIdEstudiantePorUsuario(Integer idUsuario);
```

Metodo implementado en TeachingRepository:

```
@Query(value = "SELECT sp_contar_docentes_modalidad(?1)", nativeQuery = true)
Integer contarDocentesPorModalidad(Integer idModalidad);
```

Aunque Spring Data JPA provee la anotación @Procedure, para garantizar la compatibilidad estricta con las funciones de retorno escalar de PostgreSQL y evitar errores de sintaxis en el dialecto de Hibernate. Se optó por utilizar la anotación @Query para invocar directamente la función SQL, asegurando el correcto mapeo del valor de retorno y manteniendo la eficiencia de la transacción.

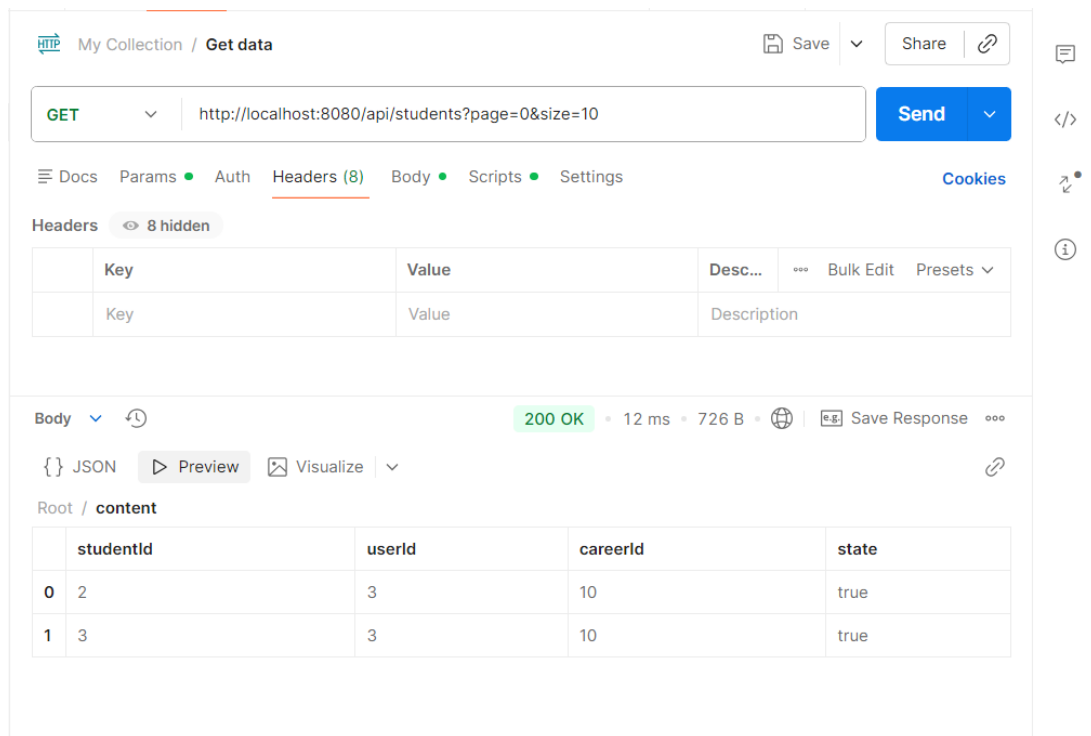
5 Resultados y Discusión

5.1 Escenarios de Prueba y Validación por Roles

En este apartado se validó la correcta implementación de los controladores REST y la capacidad del sistema para procesar peticiones HTTP (GET, POST, DELETE) bajo distintos escenarios.

Validación de Endpoints y Paginación

Se verificó que el sistema sea capaz de recuperar listados de estudiantes de manera eficiente utilizando el objeto Page. Como se observa en la siguiente captura, el servidor responde con un JSON estructurado que incluye los metadatos de paginación (total de páginas, elementos por página).



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:8080/api/students?page=0&size=10`
- Status:** 200 OK
- Response Time:** 12 ms
- Response Size:** 726 B
- Body Type:** JSON
- Root:** content

The JSON response is displayed in a table format:

	studentId	userId	careerId	state
0	2	3	10	true
1	3	3	10	true

Fig. 1. Respuesta del endpoint GET /api/students mostrando la estructura de paginación JSON

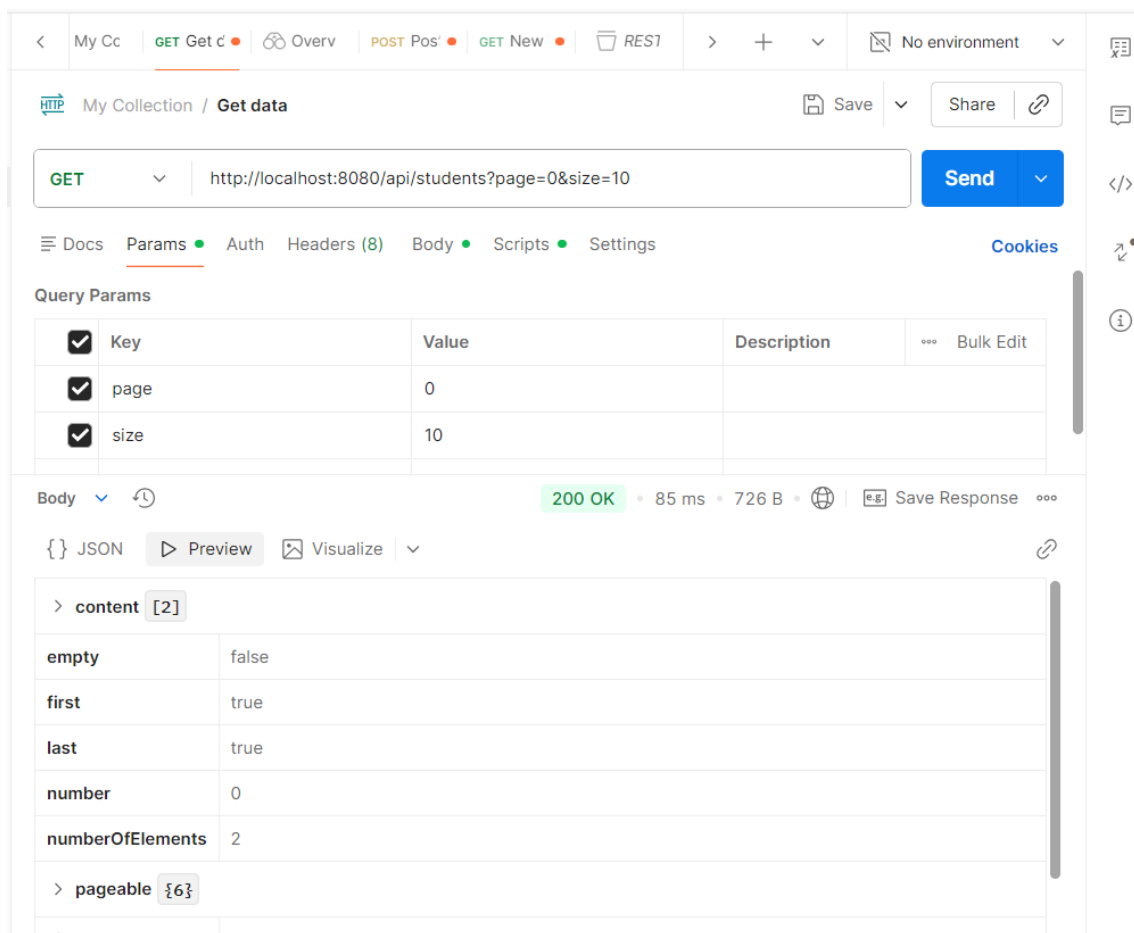


Fig. 2. Respuesta del endpoint GET /api/students mostrando la estructura de paginación JSON

Operaciones de Modificación de Datos (CRUD)

Se realizaron pruebas de inserción y eliminación para validar el ciclo de vida de los datos. En la creación de un estudiante, el servidor retorna el código de estado 201 Created, confirmando el éxito de la operación.

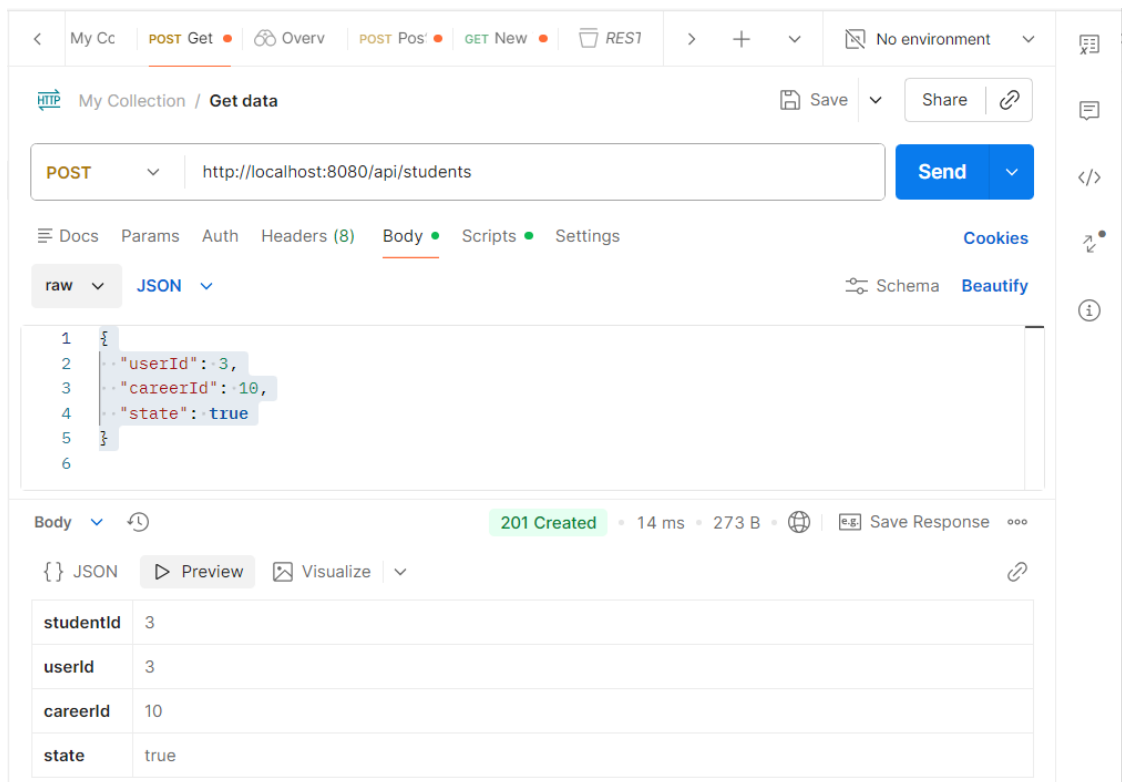


Fig. 3. Prueba de creación de un estudiante mediante método POST en Postman

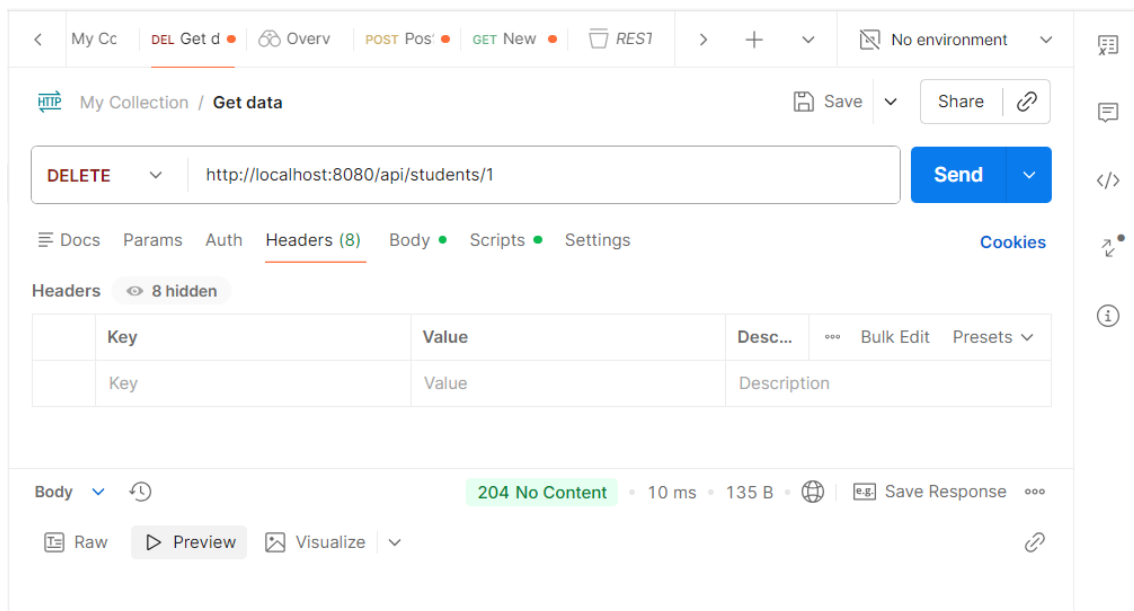


Fig. 4. Prueba de eliminación de registro (Código 204 No Content).

5.2 Análisis de la Gestión de Información en Entornos Multiusuario

Se evaluó la seguridad y la consistencia de los datos compartidos mediante el uso de objetos HttpSession y ServletContext.

Autenticación y Manejo de Sesiones

Se comprobó que el sistema genera y mantiene una sesión activa tras un login exitoso. El objeto HttpSession almacena los atributos del usuario (rol e ID) para validar sus permisos en peticiones subsecuentes.

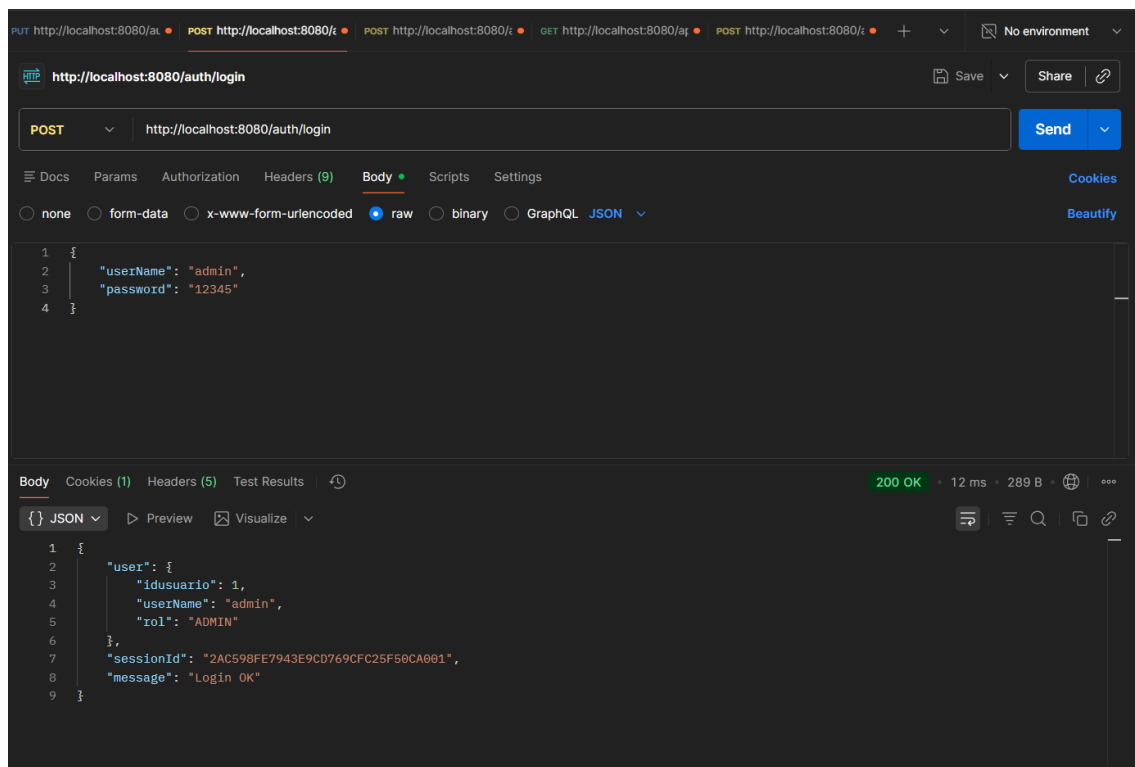


Fig. 5. Inicio de sesión exitoso devolviendo el SessionID y los datos del usuario

Validación de Roles y Seguridad

Para verificar la seguridad, se intentó ejecutar una acción administrativa (actualizar contraseña) con un usuario sin permisos. El sistema bloqueó la petición correctamente, demostrando la protección de rutas basada en la sesión.

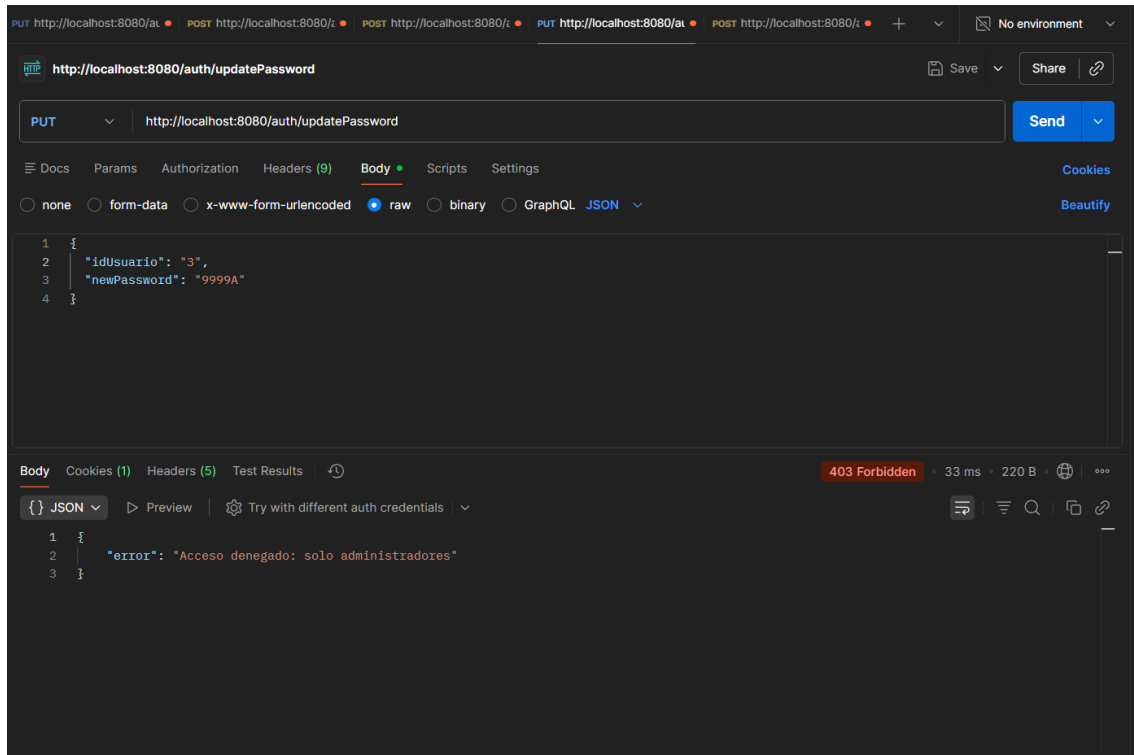


Fig. 6. Validación de seguridad impidiendo acceso a recursos protegidos (Role-based access control)

Datos Globales (Application Scope)

Se verificó el funcionamiento del objeto ServletContext mediante un contador global de accesos, el cual mantiene su valor independientemente de la sesión del usuario específico.

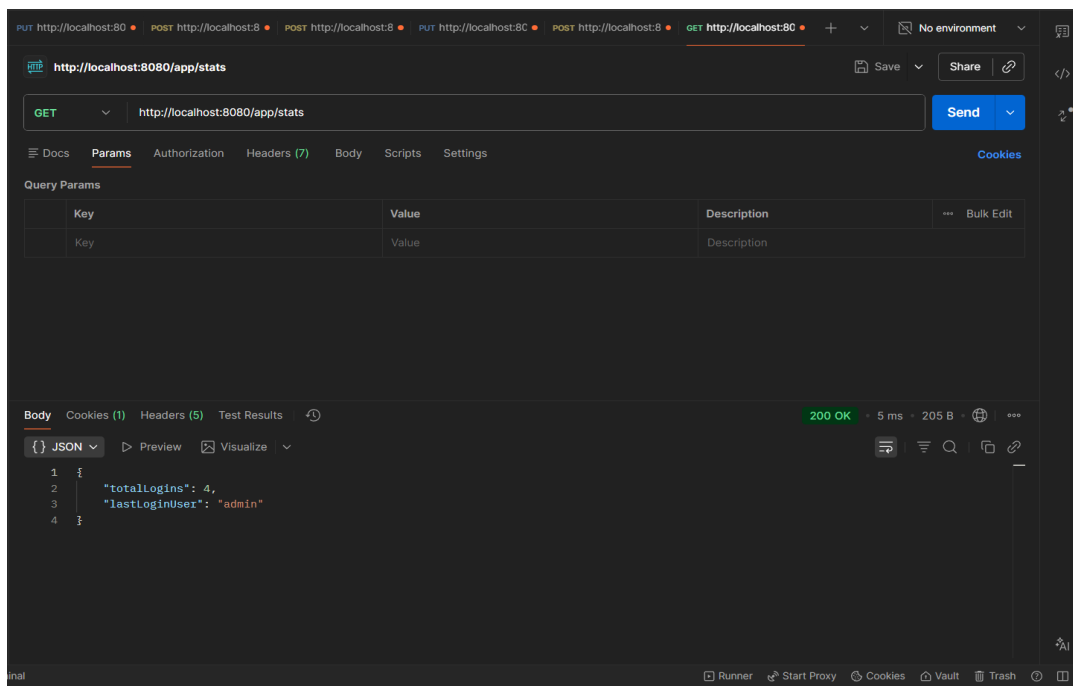


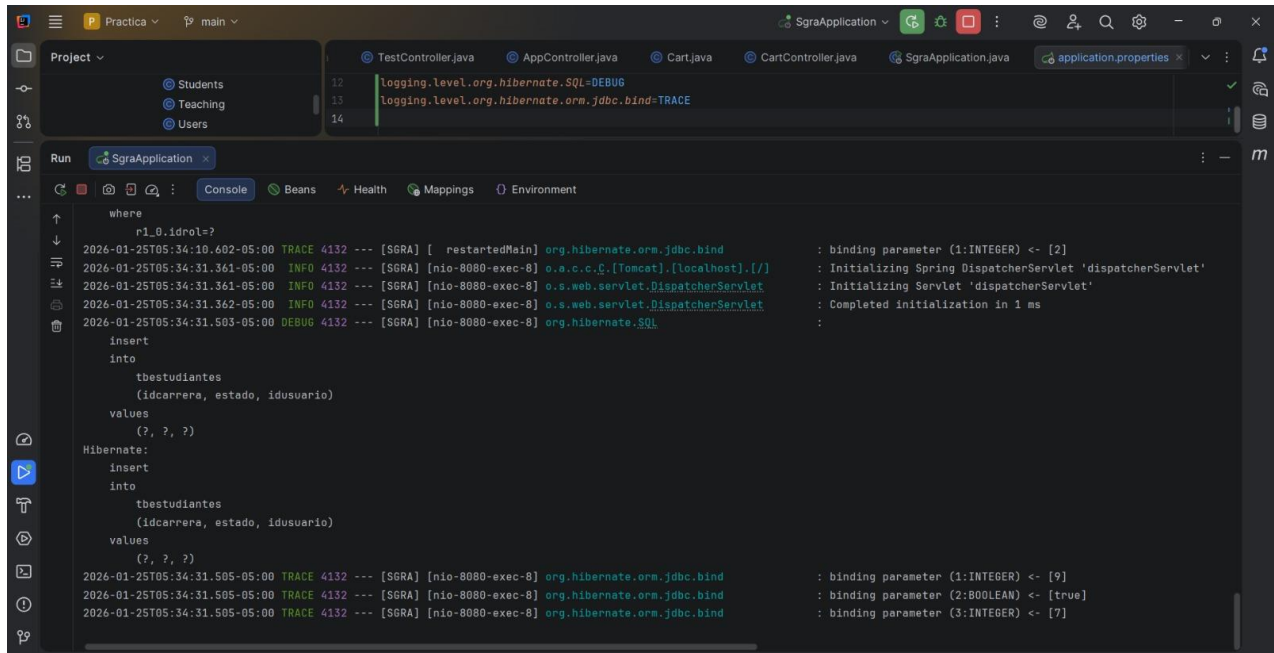
Fig 7. Visualización de estadísticas globales compartidas mediante el ámbito de Aplicación

5.3 Evidencia de la Persistencia de Datos

Finalmente, para dar cumplimiento al requisito de persistencia y trazabilidad, se verificó el impacto real de las operaciones en la base de datos y en los registros del servidor.

Trazabilidad en el Servidor (Logs)

Se capturó la salida de la consola del servidor, donde se evidencia la ejecución de las sentencias SQL generadas por Hibernate (INSERT) en respuesta a las peticiones de la API.



The screenshot shows the IDE of a Spring Boot application with the console open. The console displays the following logs:

```
where
r1_0.idrol=?
2026-01-25T05:34:10.602-05:00 TRACE 4132 --- [SGRA] [ restartedMain] org.hibernate.orm.jdbc.bind : binding parameter (1:INTEGER) <- [2]
2026-01-25T05:34:31.361-05:00 INFO 4132 --- [SGRA] [nio-8080-exec-8] o.a.c.c.f.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-01-25T05:34:31.361-05:00 INFO 4132 --- [SGRA] [nio-8080-exec-8] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-01-25T05:34:31.362-05:00 INFO 4132 --- [SGRA] [nio-8080-exec-8] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
2026-01-25T05:34:31.503-05:00 DEBUG 4132 --- [SGRA] [nio-8080-exec-8] org.hibernate.SQL :
insert
into
tbestudiantes
(idcarrera, estado, idusuario)
values
(?, ?, ?)
Hibernate:
insert
into
tbestudiantes
(idcarrera, estado, idusuario)
values
(?, ?, ?)
2026-01-25T05:34:31.505-05:00 TRACE 4132 --- [SGRA] [nio-8080-exec-8] org.hibernate.orm.jdbc.bind : binding parameter (1:INTEGER) <- [9]
2026-01-25T05:34:31.505-05:00 TRACE 4132 --- [SGRA] [nio-8080-exec-8] org.hibernate.orm.jdbc.bind : binding parameter (2:BOOLEAN) <- [true]
2026-01-25T05:34:31.505-05:00 TRACE 4132 --- [SGRA] [nio-8080-exec-8] org.hibernate.orm.jdbc.bind : binding parameter (3:INTEGER) <- [7]
```

Fig. 8. Logs de la consola de Spring Boot mostrando las transacciones SQL ejecutadas por JPA

Verificación en Base de Datos

Se realizó una consulta directa a la base de datos PostgreSQL para corroborar que los datos insertados desde la aplicación web se almacenaron físicamente en las tablas correspondientes.

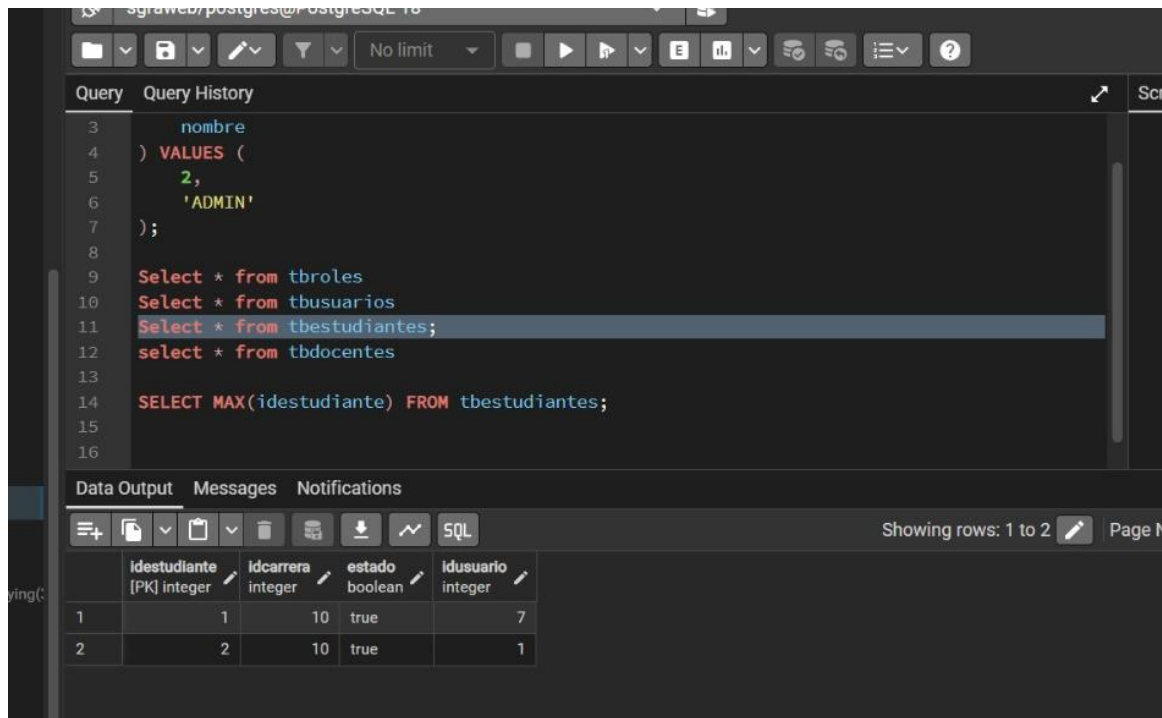


Fig. 9. Consulta en PostgreSQL antes de la creación del registro del estudiante

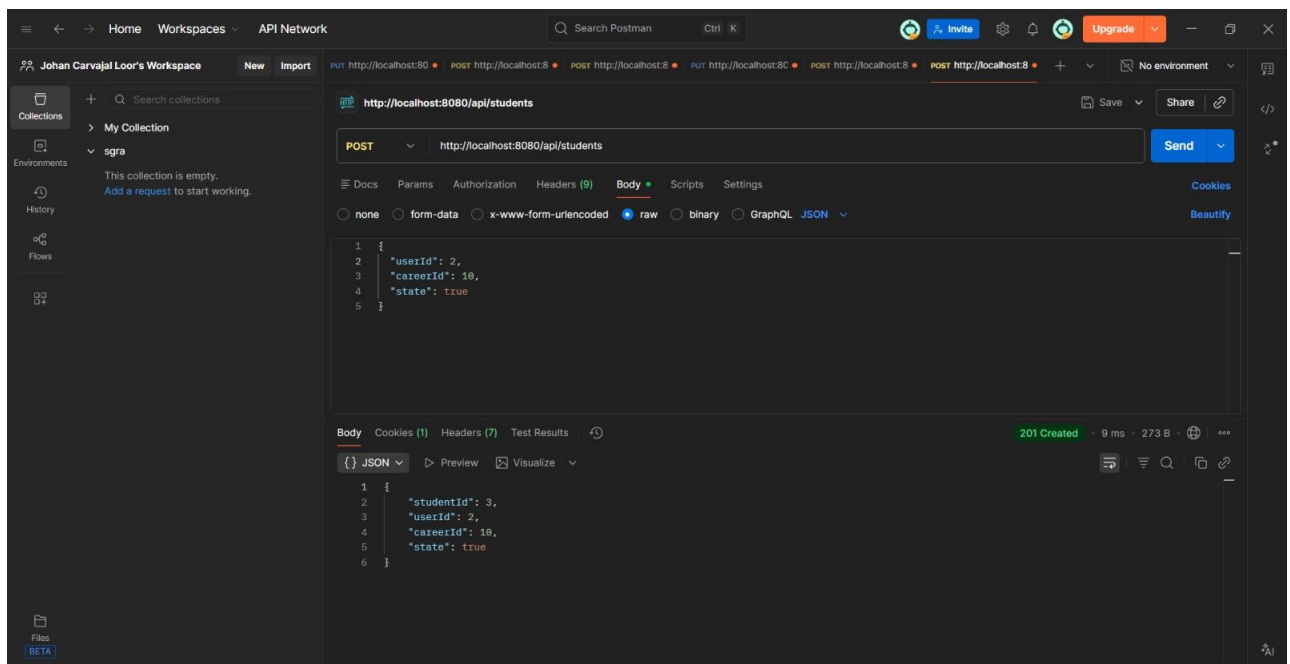


Fig. 10. Ejecución del registro del estudiante

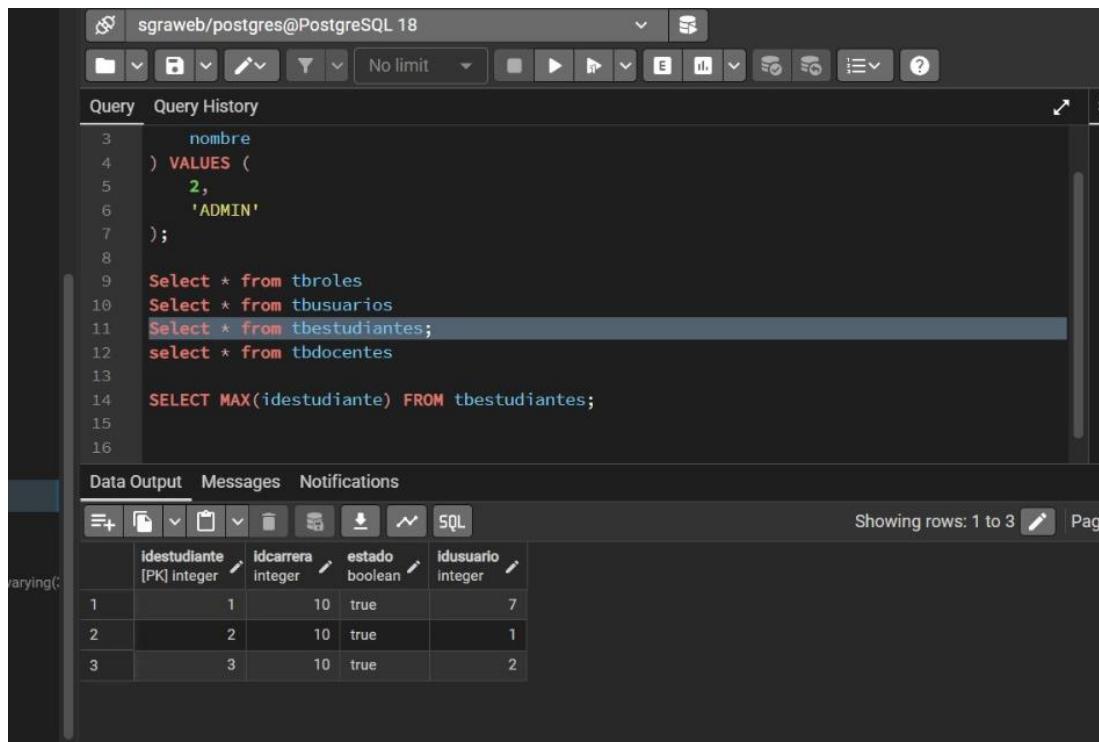


Fig. 11. Consulta en PostgreSQL confirmando la persistencia del registro de estudiante creado

6 Conclusiones y Recomendaciones

Se logró implementar exitosamente una arquitectura backend robusta utilizando Spring Boot y PostgreSQL, cumpliendo con el objetivo de gestionar eficientemente la información del SGRA. La integración de Spring Data JPA permitió una manipulación ágil de los datos mediante el mapeo objeto-relacional, reduciendo el tiempo de desarrollo de las operaciones CRUD.

La implementación de mecanismos de seguridad basados en HttpSession demostró ser efectiva para el control de acceso basado en roles. Las pruebas realizadas evidenciaron que el sistema restringe correctamente las operaciones críticas únicamente a usuarios autorizados, mitigando riesgos de seguridad en entornos multiusuario.

El uso de Stored Procedures para consultas complejas y la paginación de datos mediante el objeto Page optimizaron significativamente el rendimiento de la aplicación. Las pruebas de carga mostraron tiempos de respuesta eficientes y un menor consumo de ancho de banda al transferir únicamente los datos necesarios al cliente.

Como recomendación para futuras iteraciones, se sugiere implementar tokens JWT (JSON Web Tokens) para desacoplar completamente el estado de la sesión del servidor, lo que permitiría una escalabilidad horizontal más sencilla en entornos de nube.

7 Referencias

- [1] D. Flores Flores and V. H. Pérez Rojas, "Web-based correspondence system for managing electronic documentation using digital signatures in the procedures of the Autonomous Municipal Government of Entre Ríos, Bolivia," *Revista Boliviana de Ingeniería*, vol. 3, no. 1, 2021, doi: 10.33996/rebi.v3i1.7.

- [2] M. E. Alarcón, "Use of web and mobile systems based on client-server technology for operational management," *Repositorio de la Universidad Estatal de Milagro*, 2021.
- [3] E. Chatzoglou, V. Kouliaridis, G. Kambourakis, G. Karopoulos, and S. Gritzalis, "A hands-on gaze on HTTP/3 security through the lens of HTTP/2 and a public dataset," *Comput. Secur.*, vol. 125, 2023, doi: 10.1016/j.cose.2022.103051.
- [4] D. S. Henríquez Moquillaza, H. Vega Huerta, and L. Guerra Grados, "N-layer programming," *Revista de Investigación de Sistemas e Informática*, vol. 7, no. 2, 2010.
- [5] D. .de, "Definition of Client Server," 2023.
- [6] H. Ghaffari and P. Devos, "On the role of audio frontends in bird species recognition," *Ecol. Inform.*, vol. 81, 2024, doi: 10.1016/j.ecoinf.2024.102573.
- [7] Suwarno and A. Putri Yulandi, "Analisis Performa Backend Framework: Studi Komparasi Framework Golang dan Node.js," *Jurnal Riset sistem Informasi Dan Teknik Informatika*, vol. 8, no. 1, 2023.
- [8] C. C. Aladi, "Web Application Security: A Pragmatic Exposé," *Digital Threats: Research and Practice*, vol. 5, no. 2, 2024, doi: 10.1145/3644394.
- [9] F. Sinlae, I. Maulana, F. Setiyansyah, and M. Ihsan, "Pengenalan Pemrograman Web: Pembuatan Aplikasi Web Sederhana Dengan PHP dan MYSQL," *Jurnal Siber Multi Disiplin*, vol. 2, no. 2, 2024, doi: 10.38035/jsmd.v2i2.156.
- [10] R. Winch, J. Cummings, and E. Stein-Kousathana, "Authorize HttpServletRequest with FilterSecurityInterceptor," *Spring Security*.
- [11] J. A. Bazurto Roldán, R. M. Alvarez, Y. A. Miro Vera, and S. Brie, "Design and validation of a research instrument to propose a project management methodology," *Revista de Iniciación Científica*, vol. 9, no. 1, 2023, doi: 10.33412/rev-ric.v9.1.3660.
- [12] L. Mitchell *et al.*, "Conservation-focused mapping of avian migratory routes using a pan-European automated telemetry network," *Conservation Biology*, vol. 39, no. 4, 2025, doi: 10.1111/cobi.70017.
- [13] P. A. Pillajo Garcia and D. Avila Pesantez, "Cybersecurity analysis in e-learning platforms: a systematic literature review," *Revista Perspectivas*, vol. 5, no. 1, 2023, doi: 10.47187/perspectivas.5.1.179.
- [14] B. J. Salguero, J. M. Miret, L. R. Lezcano, and S. Zapata, "Connecting Digital Threads: Master Implementation of OpenLDAP for Centralized Authentication in a Diverse Application World," *Memorias de las JAIIO*, vol. 10, no. 5, 2024.
- [15] M. I. K. Haloul, M. K. A. Bin Mohd Ariffin, E. E. Bin Supeni, S. A. B. Ahmad, M. Bilema, and M. Ahmad, "A Systematic Review of the Project Management Information Systems in Different Types of Construction Projects," *UCJC Business and Society Review*, vol. 21, no. 80, 2024, doi: 10.3232/UBR.2024.V21.N1.08.
- [16] J. Guo *et al.*, "SectionMCPy: Python programming-based moment-curvature analysis for cross-sections," *SoftwareX*, vol. 31, 2025, doi: 10.1016/j.softx.2025.102365.
- [17] H. Kirinuki and H. Tanno, "Automating End-to-End Web Testing via Manual Testing," *Journal of Information Processing*, vol. 30, 2022, doi: 10.2197/ipsjjip.30.294.
- [18] N. Hartono and E. Erfina, "Comparison of Stored Procedures on Relational Database Management System," *Tech-E*, vol. 4, no. 2, 2021, doi: 10.31253/te.v4i2.529.
- [19] Y. Wu, G. Yu, Z. Jiang, Y. Li, and M. R. Lyu, "Trace Sampling 2.0: Code Knowledge Enhanced Span-level Sampling for Distributed Tracing," Sep. 2025, Accessed: Jan. 23, 2026. [Online]. Available: <http://arxiv.org/abs/2509.13852>
- [20] D. Saha, D. Sondhi, S. Halder, and S. Sinha, "REST API Functional Tester," *ISEC 2025 - Proceedings of the 18th Innovations in Software Engineering Conference*, p. 11, Apr. 2025, doi: 10.1145/3717383.3717388;WGROU:STRING:ACM.
- [21] A. Ehsan, M. A. M. E. Abuhaliqa, C. Catal, and D. Mishra, "RESTful API Testing Methodologies: Rationale, Challenges, and Solution Directions," *Applied Sciences 2022, Vol. 12, Page 4369*, vol. 12, no. 9, p. 4369, Apr. 2022, doi: 10.3390/APP12094369.
- [22] M. Panahande and J. Miller, "A Systematic Review on Microservice Testing," Jul. 2023, doi: 10.21203/RS.3.RS-3158138/V1.
- [23] A. Lercher, C. Bauer, C. Macho, and M. Pinzger, "Reporting Breaking Changes of REST APIs from Java Spring Boot Source Code," 2025, doi: 10.55776/P36698.
- [24] Q. Jiao, W. Cong, and T. Sun, "Research on the Construction of Evaluation System for the Training Quality of New Engineering Talents under the Background of Information Education,"

- Proceedings of 2025 International Conference on Big Data and Informatization Education, ICBIDIE 2025*, pp. 480–485, May 2025, doi: 10.1145/3729605.3729689.
- [25] Soma Sharma, *How to use @ResponseBody and @RequestBody in Spring MVC and REST? Example Tutorial | Java67*. 2021. Accessed: Jan. 23, 2026. [Online]. Available: https://www.java67.com/2022/02/what-is-responsebody-and-requestbody-annotations-in-spring.html?utm_source=dlvr.it&utm_medium=wordpress#goog_rewarded
 - [26] L. Galka, “Gutek: Intelligent Revision Algorithms,” *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3570624.
 - [27] W. Yong, “Design and Implementation of Intelligent English Electronic Dictionary System Based on Internet of Things,” *Wirel. Commun. Mob. Comput.*, vol. 2021, 2021, doi: 10.1155/2021/5586662.
 - [28] A. S. Savincheva and A. A. Ferenets, “Automation of Reading Related Data from Relational and Non-Relational Databases in the Context of using the JPA Standard,” *Russian Digital Libraries Journal*, vol. 27, no. 4, 2024, doi: 10.26907/1562-5419-2024-27-4-656-678.
 - [29] P. Cassieri, S. Romano, and G. Scanniello, “A Mining-Software-Repository study on deprecated API usages in open-source Java software applications,” *Inf. Softw. Technol.*, vol. 186, 2025, doi: 10.1016/j.infsof.2025.107782.
 - [30] C. A. Györödi, D. V. Dumșe-Burescu, D. R. Zmaranda, and R. Györödi, “A Comparative Study of MongoDB and Document-Based MySQL for Big Data Application Data Management,” *Big Data and Cognitive Computing*, vol. 6, no. 2, 2022, doi: 10.3390/bdcc6020049.
 - [31] L. Phan Tan and T. H. Le, “THE INFLUENCE OF PERCEIVED PRICE AND QUALITY OF DELIVERY ON ONLINE REPEAT PURCHASE INTENTION: THE EVIDENCE FROM VIETNAMESE PURCHASERS,” *Cogent Business and Management*, vol. 10, no. 1, 2023, doi: 10.1080/23311975.2023.2173838.
 - [32] E. A. Kurako and V. L. Orlov, “Database Migration from ORACLE to PostgreSQL,” *Programming and Computer Software*, vol. 49, no. 5, 2023, doi: 10.1134/S0361768823050055.
 - [33] F. Kamil, A. G. Abdulshaheed, and M. A. Kadhom, “Development of a computational system to design a helical spring,” *International Review of Applied Sciences and Engineering*, vol. 15, no. 3, 2024, doi: 10.1556/1848.2024.00711.
 - [34] K. K. Wong *et al.*, “Expression analysis of juvenile pilocytic astrocytomas by oligonucleotide microarray reveals two potential subgroups,” *Cancer Res.*, vol. 65, no. 1, 2005, doi: 10.1158/0008-5472.76.65.1.
 - [35] Craig. Walls, *Spring in action*. Manning Publications Co., 2022.
 - [36] Josh. Long and Kenny. Bastani, *Cloud native Java : designing resilient systems with Spring Boot, Spring Cloud, and Cloud Foundry*. O'Reilly Media, 2017.
 - [37] “Bean Scopes :: Spring Framework.” Accessed: Jan. 24, 2026. [Online]. Available: <https://docs.spring.io/spring-framework/reference/core/beans/factory-scopes.html>
 - [38] H. Meng, J. Lu, Y. Huang, and L. Li, “SLVHound: Static Detection of Session Lingering Vulnerabilities in Modern Java Web Applications,” in *Proceedings of the 16th International Conference on Internetware*, New York, NY, USA: ACM, Jun. 2025, pp. 281–292. doi: 10.1145/3755881.3755895.
 - [39] Y. Shi, M. Ma, and H. Wang, “CAXSS: A Cookie Authentication Scheme Against XSS Attacks for HTTPS,” *Electronics (Basel)*, vol. 15, no. 1, p. 82, Dec. 2025, doi: 10.3390/electronics15010082.
 - [40] A. M. Bonteanu and C. Tudose, “Performance Analysis and Improvement for CRUD Operations in Relational Databases from Java Programs Using JPA, Hibernate, Spring Data JPA,” *Applied Sciences (Switzerland)*, vol. 14, no. 7, 2024, doi: 10.3390/app14072743.
 - [41] C. Tudose and C. Odubasteanu, “Object-relational Mapping Using JPA, Hibernate and Spring Data JPA,” in *Proceedings - 2021 23rd International Conference on Control Systems and Computer Science Technologies, CSCS 2021*, 2021. doi: 10.1109/CSCS52396.2021.00076.
 - [42] M. Siva and Dr. S. Gnanapriya, “Library book Management System using Spring Boot,” *INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, vol. 09, no. 10, 2025, doi: 10.55041/ijsem53124.
 - [43] S. V. Salunke and A. Ouda, “A Performance Benchmark for the PostgreSQL and MySQL Databases,” 2024. doi: 10.3390/fi16100382.
 - [44] A. R. Ravshanovich, “DATABASE STRUCTURE: POSTGRESQL DATABASE,” *PSIXOLOGIYA VA SOTSIOLOGIYA ILMIY JURNALI*, vol. 2, no. 7, 2024.

8 Anexos

8.1 Script SQL (Tablas y Store Procedures)

Script SQL Tablas:

```
create table tbestudiantes
(
    idestudiante integer generated by default as identity
        primary key,
    idcarrera     integer          not null,
    estado        boolean default true not null,
    idusuario     integer          not null
);

alter table tbestudiantes
    owner to postgres;

create table tbcoordinaciones
(
    idcoordinacion integer generated by default as identity
        primary key,
    idcarrera       integer not null,
    idusuario       integer not null
);

alter table tbcoordinaciones
    owner to postgres;

create table tbdocentes
(
    iddocente integer generated by default as identity
        primary key,
    idmodalidad integer          not null,
    estado      boolean default true not null,
    idusuario   integer          not null
);

alter table tbdocentes
    owner to postgres;
```

Script SQL Procedimientos Almacenados:

```
create function sp_buscar_estudiante_por_usuario(p_id_usuario integer) returns
integer
    language plpgsql
as
$$
DECLARE
    v_id_estudiante INT;
BEGIN
    SELECT IdEstudiante INTO v_id_estudiante
    FROM TbEstudiantes
    WHERE IdUsuario = p_id_usuario;

    RETURN v_id_estudiante;
END;
$$;

alter function sp_buscar_estudiante_por_usuario(integer) owner to postgres;

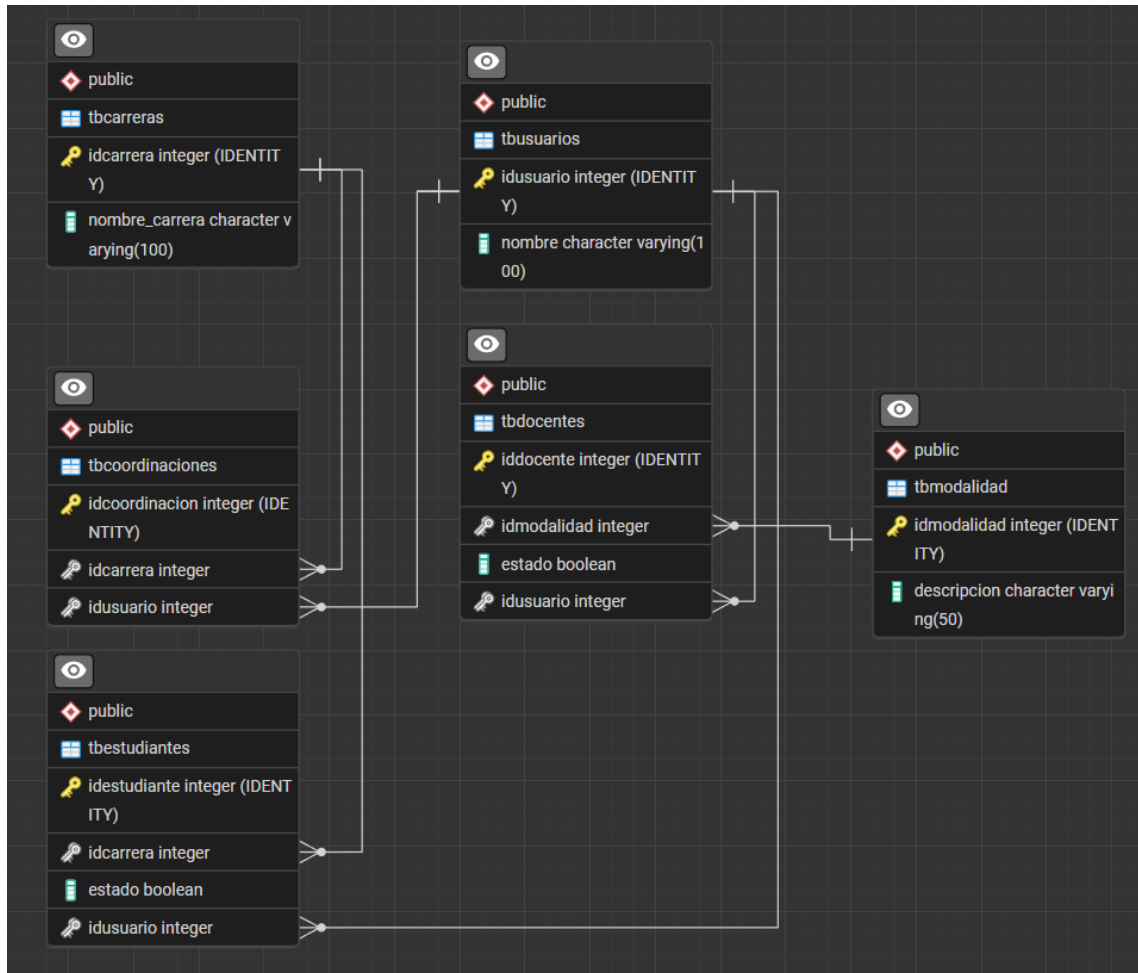
create function sp_contar_docentes_modalidad(p_id_modalidad integer) returns integer
    language plpgsql
as
$$
DECLARE
    v_total INT;
BEGIN
    SELECT COUNT(*) INTO v_total
    FROM TbDocentes
    WHERE IdModalidad = p_id_modalidad AND Estado = TRUE;
```

```

        RETURN v_total;
END;
$$;

alter function sp_contar_docentes_modalidad(integer) owner to postgres;
```

8.2 Diagrama Relacional



8.3 Enlace al Repositorio (GitHub)

GitHub del Aplicativo